

Assignment #04

Distributed Programming

Design and implementation report

Academic year 2025-2026 - Concurrent and Distributed Programming

This report documents the submitted implementation and its concurrency choices. The design follows the terminology and principles in merged.pdf and the assignment README.

1. Exercise 1 - Clustered Smart Home Alarm

The alarm state machine from Assignment 03 is deployed in an Apache Pekko Cluster. `application.conf` selects the cluster actor provider and two seed addresses. `ClusterNode` accepts a port and one of three roles: controller, sensor or keypad. The provided commands start at least three JVM nodes; the third may use a dynamically assigned port.

Sensors retain a unique `sensorId` and a type (motion or door/window). Sensors and keypads use a remote `ActorSelection` and communicate with the control actor only through serializable messages. No shared object crosses a node boundary. Exit and entry timing remains local to the controller actor, so state transitions are serialized by its mailbox.

Safe recovery

A volatile actor cannot know whether the lost pre-restart state was Disarmed, Armed or in a delay. Therefore `preStart` always selects `RECOVERY`. In this state sensor events are logged and ignored. A correct PIN is the only event that establishes `DISARMED`. This explicitly satisfies the required fail-safe behavior and avoids the unsafe assumption that a recreated controller is disarmed.

2. Exercise 2 - Distributed Tic-Tac-Toe with RMI

The server exports a `GameLobby` remote object. `createGame` atomically reserves a name and returns a newly exported `RemoteGame`; `joinGame` locates the named game, atomically assigns O and returns the same remote object. This matches the required create/wait and join-by-name workflows.

Remote operation	Concurrency and validation
<code>createGame</code>	<code>ConcurrentHashMap.putIfAbsent</code> prevents duplicate names
<code>join</code>	Synchronized; exactly one opponent can claim O
<code>makeMove</code>	Synchronized; validates identity, turn, bounds and empty cell atomically
<code>snapshot</code>	Synchronized defensive immutable copy; safe for polling clients
<code>waitingGames</code>	Lists only games whose state is <code>WAITING</code>

The game remote object owns its board, players, turn and lifecycle state. It recognizes all rows, columns and diagonals, switches turns only after a legal move, and terminates in `X_WON`, `O_WON` or `DRAW`. Remote failures are reported as `RemoteException` without partially applying an invalid operation.

3. Verification and operation

The RMI unit test joins two players, interleaves legal moves and verifies an X row victory. `mvn test` compiles both exercises and runs the test. For an end-to-end cluster check, start controller on 2551, sensor on 2552 and keypad on port 0 as documented in `README-SOLUTION.md`. For RMI, start `TicTacToeServer` and connect two `TicTacToeClient` processes using the same game name.